CPE412 Pattern Recognition

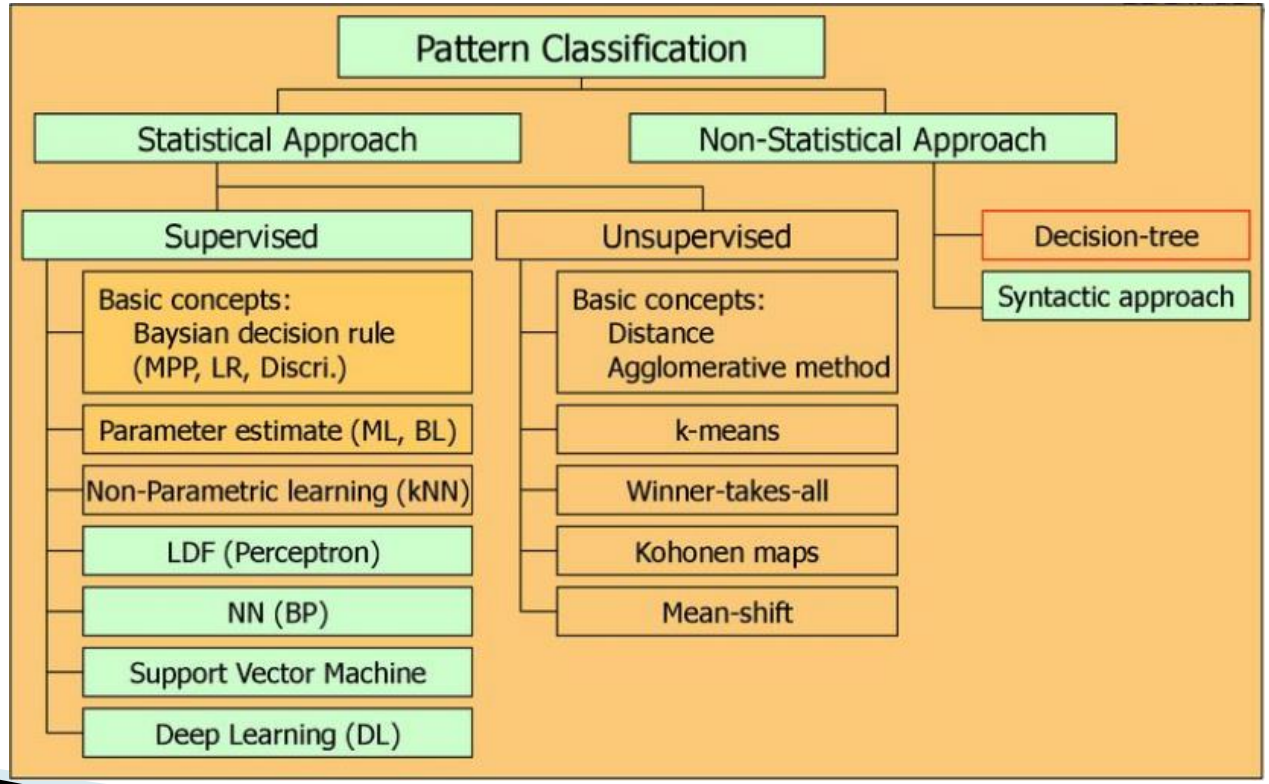
Week 11

Decision Tree

Dr. Nehad Ramaha,
Computer Engineering Department
Karabük Universities

The class notes are a compilation and edition from many sources. The instructor does not claim intellectual property or ownership of the lecture notes.

Overview

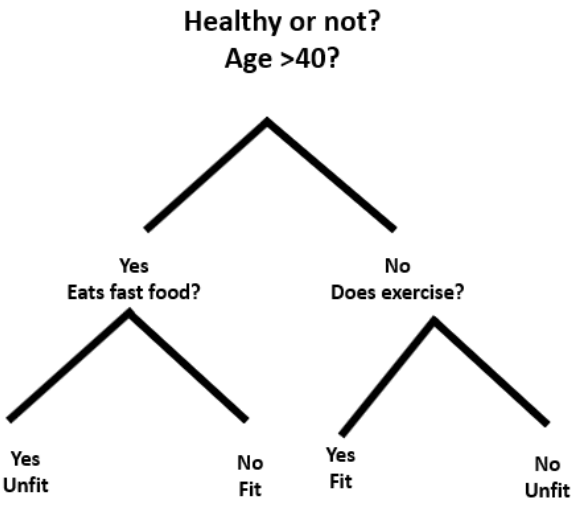

Pattern Classification
Statistical Approach
Non-Statistical Approach
Supervised
Unsupervised
Decision-tree
Syntactic approach
Basic concepts:
Baysian decision rule
(MPP, LR, Discri.)
Parameter estimate (ML, BL)
Non-Parametric learning (kNN)
LDF (Perceptron)
NN (BP)
Support Vector Machine
Deep Learning (DL)
Basic concepts:
Distance
Agglomerative method
k-means
Winner-takes-all
Kohonen maps
Mean-shift

Decision Tree

- Most pattern recognition methods address problems where there is a natural measure of distance between feature vectors.
- What happens when the classification problem involves nominal (categorical) data, e.g., descriptions that are discrete and without any natural notion of similarity or even ordering?
- A common representation for this kind of data is a list of attributes (instead of a vector of real numbers).

Decision Tree

- It is natural and intuitive to classify a pattern through a sequence of questions, in which the next question asked depends on the answer to the current question.
- Such a sequence of questions can be represented as a decision tree.
- In a decision tree, the top node is called the root node, and is connected by directional links (branches) to other nodes.
- These nodes are similarly connected until terminal (leaf) nodes, which have no further links, are reached.



Decision Tree

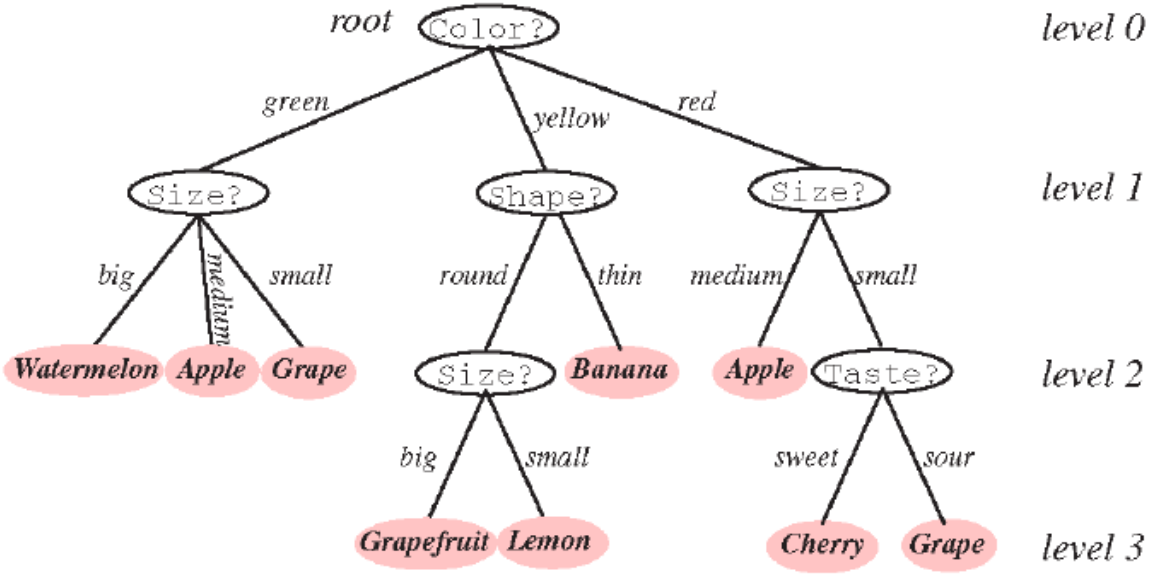


Figure 3: An example decision tree that uses the attributes {color, shape, taste, size}. Note that the same question can appear in different places in the tree and that different questions can have different numbers of branches. Moreover, different leaf nodes can be labeled by the same class.

Decision Tree

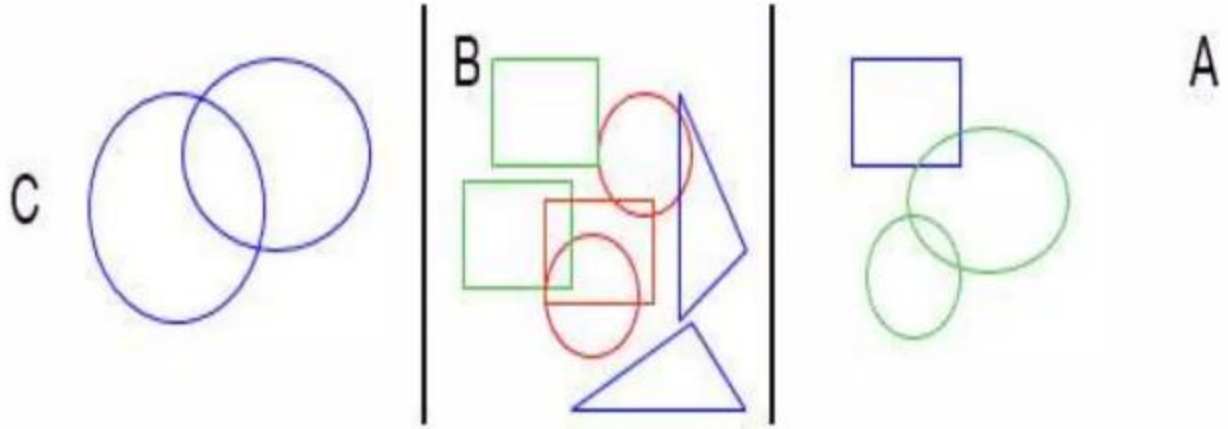
- Decision tree is a simple but powerful learning Paradigm.
- It is a type of classification algorithm for supervised learning.
- A decision tree is a tree in which each branch node represents a choice between a number of alternatives and each leaf node represents a decision.
- A node with outgoing edges is called an internal or test node. All other nodes are called leaves (also known as terminal or decision nodes).

Construction of the Decision Tree

1) First test all attributes and select the one that will function as the best root.
2) Break-up the training set into subsets based on the branches of the root node.
3) Test the remaining attributes to see which ones fit best underneath the branches of the root node.
4) Continue this process for all other branches until
    I. All the examples of a subset are of one type.
    II. There are no examples left.
    III. There are no more attributes left.

Decision Tree Example

- Given a collection of shape and colour example, learn a decision tree that represents it. Use this representation to classify new examples.



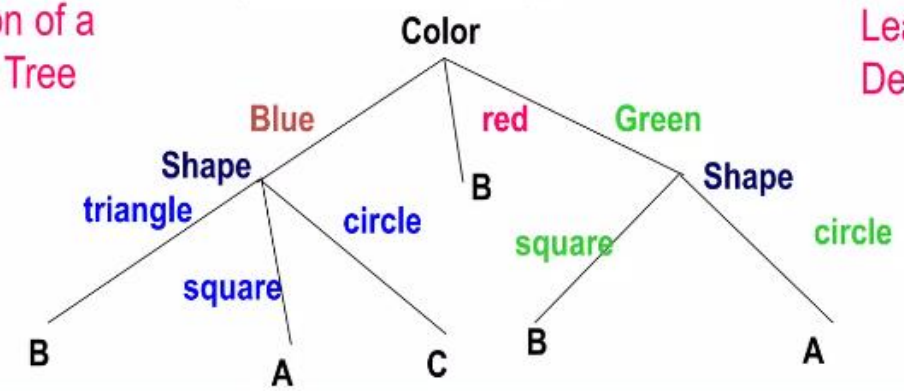
Decision Tree Example

- Decision Trees are classifiers for instances represented as features vectors. (color = ;shape = ;label =)
- Nodes are tests for feature values;
- There is one branch for each value of the feature
- Leaves specify the categories (labels)
- Can categorize instances into multiple disjoint categories – multi-class

(color = RED ;shape = triangle)

Evaluation of a Decision Tree

Learning a Decision Tree



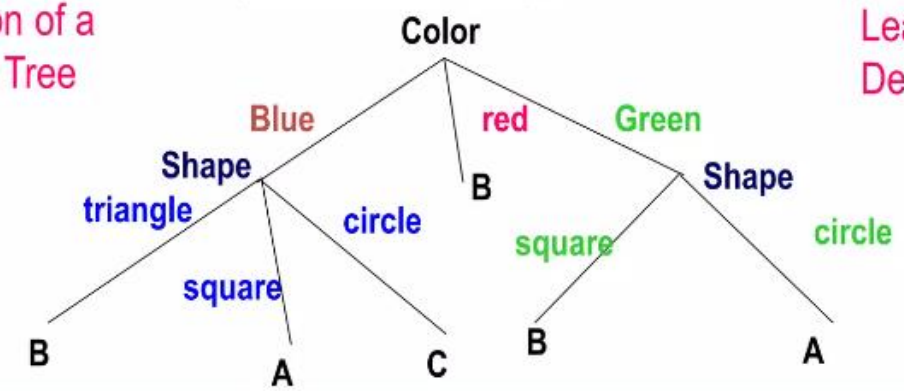
Decision Tree Example

- Decision Trees are classifiers for instances represented as features vectors. (color = ;shape = ;label =)
- Nodes are tests for feature values;
- There is one branch for each value of the feature
- Leaves specify the categories (labels)
- Can categorize instances into multiple disjoint categories – multi-class

(color = RED ;shape = triangle)

Evaluation of a Decision Tree

Learning a Decision Tree



Advantages of Decision Tree

- Easy to understand and interpret, making them accessible to non-experts.
- Handle both numerical and categorical data without requiring extensive preprocessing.
- Provides insights into feature importance for decision-making.
- Handle missing values and outliers without significant impact.
- Applicable to both classification and regression tasks.

Disadvantages of Decision Tree

- Disadvantages include the potential for overfitting
- Sensitivity to small changes in data, limited generalization if training data is not representative
- Potential bias in the presence of imbalanced data.

Learning Decision Trees

- From a training set, many possible trees
- Do not learn any tree but the simplest one! (simplicity for generalization)
- The simplest tree is the shortest one.
- One approach: Creating all the trees and keep the shortest one... But this approach requires a too much computation time
- ID3/C4.5 builds short decision trees more efficiently(although does not ensure to get the shortest one).

Attribute Selection Measures:

- Construction of Decision Tree: A tree can be “learned” by splitting the source set into subsets based on Attribute Selection Measures.
- Attribute selection measure (ASM) is a criterion used in decision tree algorithms to evaluate the usefulness of different attributes for splitting a dataset.
- The goal of ASM is to identify the attribute that will create the most homogeneous subsets of data after the split, thereby maximizing the **information gain**.
- This process is repeated on each derived subset in a recursive manner called recursive partitioning. The recursion is completed when the subset at a node all has the same value of the target variable, or when splitting no longer adds value to the predictions.

Entropy

- entropy is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information.
- In order to define Information Gain precisely, we need to discuss Entropy first.
- A formula to calculate the homogeneity of a sample.
- A completely homogeneous sample has entropy of 0 (leaf node).
- An equally divided sample has entropy of 1.
- The formula for entropy is:

$$\text{Entropy}(S) = -p(I)\log_2 p(I)$$

- where p(I) is the proportion of S belonging to class I. ∑ is over total outcomes.

Entropy

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

- Example 1

If S is a collection of 14 examples with 9 YES and 5 NO examples then

0.643 x -0.637 0.357 x -1.486

Entropy(S) = - (9/14) Log_2 (9/14) - (5/14) Log_2 (5/14)

= 0.940 0.410 + 0.530

Information Gain

- The information gain is based on the decrease in entropy after a dataset is split on an attribute.
- The formula for calculating information gain is:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum((|S_v| / |S|) * \text{Entropy}(S_v))$$

Where, S_v = subset of S for which attribute A has value v

- $|S_v|$ = number of elements in S_v
- $|S|$ = number of elements in S

if we have sub groups

The ID3 algorithm

- The calculation for information gain is the most difficult part of this algorithm.
- ID3 performs a search whereby the search states are decision trees and the operator involves adding a node to an existing tree.
- It uses information gain to measure the attribute to put in each node, and performs a greedy search using this measure of worth. The algorithm goes as follows:
- Given a set of examples, S, categorized in categories ci, then:
  - 1. Choose the root node to be the attribute, A, which scores the highest for information gain relative to S.
  - 2. For each value v that A can possibly take, draw a branch from the node.
  - 3. For each branch from A corresponding to value v, calculate Sv. Then:
    - If Sv is empty, choose the category $c_{default}$ which contains the most examples from S, and put this as the leaf node category which ends that branch.
    - If Sv contains only examples from a category c, then put c as the leaf node category which ends that branch.
    - Otherwise, remove A from the set of attributes which can be put into nodes. Then put a new node in the decision tree, where the new attribute being tested in the node is the one which scores highest for information gain relative to Sv (note: not relative to S). This new node starts the cycle again (from 2), with S replaced by Sv in the calculations and the tree gets built iteratively like this.

Example

- Suppose we want to train a decision tree using the following instances:

Weekend (Example)	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay in
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Example

- The first thing we need to do is work out which attribute will be put into the node at the top of our tree: either weather, parents or money. To do this, we need to calculate:

$$
\begin{aligned}
\text{Entropy}(S) &= -p_{cinema}\log_2(p_{cinema}) - p_{tennis}\log_2(p_{tennis}) - p_{shopping}\log_2(p_{shopping}) - p_{stay_in}\log_2(p_{stay_in}) \\
&= -(6/10) * \log_2(6/10) - (2/10) * \log_2(2/10) - (1/10) * \log_2(1/10) - (1/10) * \log_2(1/10) \\
&= -(6/10) * -0.737 - (2/10) * -2.322 - (1/10) * -3.322 - (1/10) * -3.322 \\
&= 0.4422 + 0.4644 + 0.3322 + 0.3322 = 1.571
\end{aligned}
$$

Example

and we need to determine the best of:

Note: S has 4 categories(Cinema, Tennis, Stay in, Shopping). To calculate for Example

Entropy(S_{sun})= Σ- p_i(of sun with each category of S/no of sun)$\log_2 p_i$

Entropy(S_{sun})= - (1/3) $\log_2$ (1/3) (2/3) $\log_2$ (2/3) - (0/3) $\log_2$ (0/3) - (0/3) $\log_2$ (0/3)

1 sun with Cinema

2 sun with Tennis

0 sun with stay in and shopping

$-0.333 \log_2(0.333) - 0.666 \log_2(0.666) - 0$

$$\text{Gain}(S, \text{weather}) = 1.571 - (|S_{sun}|/10)*\text{Entropy}(S_{sun}) - (|S_{wind}|/10)*\text{Entropy}(S_{wind}) - (|S_{rain}|/10)*\text{Entropy}(S_{rain})$$
$$= 1.571 - (0.3)*\text{Entropy}(S_{sun}) - (0.4)*\text{Entropy}(S_{wind}) - (0.3)*\text{Entropy}(S_{rain})$$
$$= 1.571 - (0.3)*(\underline{0.918}) - (0.4)*(0.81125) - (0.3)*(0.918) = 0.70$$

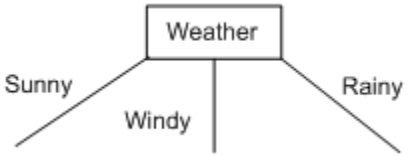
$$\text{Gain}(S, \text{parents}) = 1.571 - (|S_{yes}|/10)*\text{Entropy}(S_{yes}) - (|S_{no}|/10)*\text{Entropy}(S_{no})$$
$$= 1.571 - (0.5) * 0 - (0.5) * 1.922 = 1.571 - 0.961 = 0.61$$

$$\text{Gain}(S, \text{money}) = 1.571 - (|S_{rich}|/10)*\text{Entropy}(S_{rich}) - (|S_{poor}|/10)*\text{Entropy}(S_{poor})$$
$$= 1.571 - (0.7) * (1.842) - (0.3) * 0 = 1.571 - 1.2894 = 0.2816$$

Example

- This means that the first node in the decision tree will be the weather attribute. As an exercise, convince yourself why this scored (slightly) higher than the parents attribute – remember what entropy means and look at the way information gain is calculated.

- From the weather node, we draw a branch for the values that weather can take: sunny, windy and rainy:

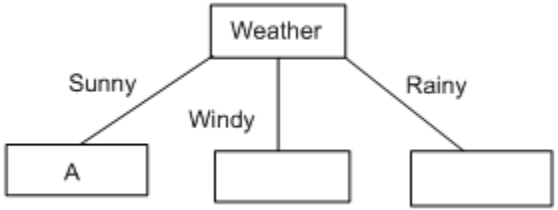


Example

- Now we look at the first branch. S_{sunny} = {W1, W2, W10}.
- This is not empty, so we do not put a default categorization leaf node here.
- The categorizations of W1, W2 and W10 are Cinema, Tennis and Tennis respectively. As these are not all the same, we cannot put a categorization leaf node here.
- Hence we put an attribute node here, which we will leave blank for the time being.

Example

- Looking at the second branch, S_{windy} = {W3, W7, W8, W9}.
- Again, this is not empty, and they do not all belong to the same class, so we put an attribute node here, left blank for now.
- The same situation happens with the third branch, hence our amended tree looks like this:



Example

- Now we have to fill in the choice of attribute A, which we know cannot be weather, because we've already removed that from the list of attributes to use. So, we need to calculate the values for Gain(S_{sunny}, parents) and Gain(S_{sunny}, money). Firstly, Entropy(S_{sunny}) = 0.918.
- Next, we set S to be S_{sunny} = {W1,W2,W10} (and, for this part of the branch, we will ignore all the other examples). In effect, we are interested only in this part of the table:

Weekend (Example)	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

Example

- Hence we can calculate:

only 1 category (cinema)

($\log_2 1 = 0$) $1/1$ → we have 1 yes

$2/2 \cdot \log_2 2/2$

$$\text{Gain}(S_{sunny}, \text{parents}) = 0.918 - (|S_{yes}|/|S|)*\text{Entropy}(S_{yes}) - (|S_{no}|/|S|)*\text{Entropy}(S_{no})$$
$$= 0.918 - (1/3)*0 - (2/3)*0 = 0.918$$

$$\text{Gain}(S_{sunny}, \text{money}) = 0.918 - (|S_{rich}|/|S|)*\text{Entropy}(S_{rich}) - (|S_{poor}|/|S|)*\text{Entropy}(S_{poor})$$
$$= 0.918 - (3/3)*0.918 - (0/3)*0 = 0.918 - 0.918 = 0$$

1 category with cinema

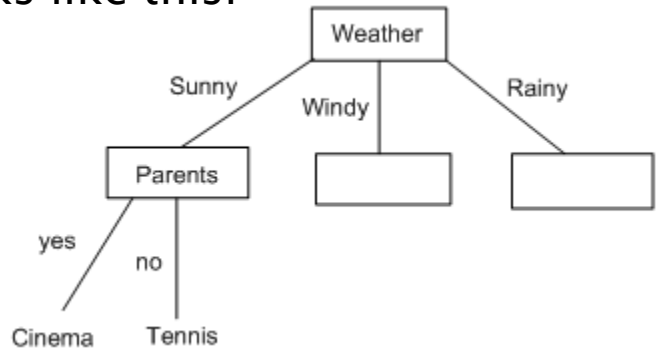
2 category with (tennis)

$-1/3 \cdot \log_2 1/3 \quad -2/3 \cdot \log_2 \frac{2}{3} =$

3 rich → rich

Example

- Given our calculations, attribute A should be taken as parents
- The two values from parents are yes and no, and we will draw a branch from the node for each of these.
- looking at S_{yes}, we see that the only example of this is W1. Hence, the branch for yes stops at a categorization leaf, with the category being Cinema.
- Also, S_{no} contains W2 and W10, but these are in the same category (Tennis). Hence the branch for no ends here at a categorization leaf(Tennis).
- Hence our upgraded tree looks like this:



Finishing this tree off is left as an exercise.

Thanks ☺